

# Assumptions for Entity and Relationship Model

## Primary Assumptions

We have developed the entity relationships for the application with a few primary assumptions. We first assume that what users create is isolated from other users, this includes and is not limited to WorkoutTemplates Entities, Exercises Entities, and Sets. Users can freely modify what they have created (such as exercises and WorkoutTemplates) and can freely interact with user-related entities and attributes.

Additionally, we made multiple design decisions that were cognizant of the default exercises we are providing users. We chose to make it so users cannot modify the default exercises we provide them. Additionally, by storing user related data in separate entities from the Exercises we made it so we won't run into any user modification or user data issues when sharing the default exercises among all of the users. In making these decisions, we have greatly reduced the comparative size of our dataset for "x" number of users and therefore reduced storage costs because we do not need to make copies of all of the default exercises for every new user.

## Entity Relationships

The Users entity has an exactly one Users to many relationship with the WorkoutTemplates entity. The reason for this is that we want the users to have individual WorkoutTemplates that are not shared across users so it therefore wouldn't make sense for a WorkoutTemplate to exist if a user doesn't exist to perform the workout. The relationship is a "owns" relationship because the User entity can create, modify, and delete the WorkoutTemplates that are tied to their account (because they own them). This relationship is a weak relationship because if a user deletes their account we will delete the WorkoutTemplates because there is no reason for a WorkoutTemplate to exist if a user doesn't exist to perform the workout. We could not merge the WorkoutTemplates entity or any of its attributes inside of the Users entity because, in doing so, we would be unable to represent more than one WorkoutTemplates entity in our web application (or we would be needlessly limited to whatever number of columns we set aside for the WorkoutTemplate attributes in the Users entity).

As we previously mentioned with our primary assumptions, we did not want to have to copy the default exercise records for every new user (because we would be duplicating thousands of records) so we needed to separate user related actions from the Exercises entity. We additionally wanted the Users entity to have the ability to create and modify the exercises that they have created. As a result, we added the "ownerId" attribute to the Exercises entity which is null if an exercise is a default exercise but contains the Users userId if the user created the exercise. We described this zero to one Users to many Exercises relationship "owns" because a user will have ownership of the exercises they create (and thus can create, modify, and delete the exercises). Since no other users have access to the non-default exercises, each user can freely delete or modify the exercises they create without affecting any other users. We were unable to merge the Exercises entity inside of the Users entity because if we did so we

would be limited to the number of exercises we could store (determined by the number of columns we set aside) which would severely limit our web applications usefulness. We similarly could not merge the Users entity inside of the Exercises entity by setting aside a certain number of columns because in doing so we would put a very limiting upper threshold on the number of users who would be able to use our application. The same primary assumption caused the developers to make certain decisions on the relationship between the Users, Exercises, and ExerciseLog Entities as well.

The Users entity has a many to many relationship with the Exercises entity through the many to many relationships that both entities have with the ExerciseLog entity. The relationship between the entities was specifically set up in this manner because we wanted to separate the user data from the Exercises entity so users can share default exercises without having to replicate them for new users (otherwise the user data stored in the Exercises table would be incorrectly modified by other users). We wanted to avoid this unneeded data replication because otherwise we would've had to copy thousands of exercises for every new user which would violate one of the application constraints we discussed earlier. Additionally, we wished to keep track of specific interactions the Users entity has with the Exercises entity which needed to be accomplished through the ExerciseLog entity because the Users entity has a many to many relationship with the Exercises entity (so we couldn't store these attributes in either table). Finally, we decided to name the ExerciseLog entity as such since it is used to retrieve the most recent interaction a user had (in regards to what sets were last performed) for each exercise.

The WorkoutTemplates entity has a many to many relationship with the Exercises entity which is represented by the "contains" relationship. This describes what exercises a WorkoutTemplate contains and in what order they should be displayed in when the user views the workout page. In our Logical Design, we represented this "contains" relationship with a table called the WorkoutContents table. Since we wished to keep track of the order in which each exercise should be displayed on the workout webpages (which are created from a chosen WorkoutTemplates record and connected entities) we needed to keep track of this attribute outside of the WorkoutTemplates entity. We could not simply place the order attribute inside of the Exercises entity because in doing so that would make it so each exercise would have to appear in a certain order no matter what the workout is. This wouldn't make sense because the ordering is relative to the other exercises that are inside of the WorkoutTemplate and differ for each WorkoutTemplate record so different exercises could have the same order (such as an order of 2) due to being given that order in different WorkoutTemplates which would cause a conflict if they were added to a third WorkoutTemplate record. Therefore, we needed to create a new entity called "WorkoutContents" which would contain the ordering for every exercise in every WorkoutTemplate.

The ExerciseLog entity has an exactly one to many relationship with the Sets entity because this relationship demonstrates how the given exercise was most recently performed. We plan to use this information to auto-fill workouts that are generated from WorkoutTemplates so the user doesn't have to remember how they last performed a given exercise (in regards to how many reps and how much weight they used the last time they performed it for example).

Given this line of thinking, it wouldn't make sense for set information to exist if the exercise was never performed in the first place (since it represents what how the user last performed the exercise), as a result we modeled the "complete" relationship in the entity relationship diagram as a weak exactly one ExerciseLog to many sets relationship. We could not merge the Sets entity into the ExerciseLog entity because if we did so we would be limited to the number of sets a user could perform for an exercise (determined by the number of columns we set aside) which would severely limit our web applications usefulness. By having a weak exactly one to many relationship, a user can perform/log as many sets as they wish to for a given exercise.

The Exercises entity has a many to many relationship with the Muscles entity. We explicitly chose to model Muscles as an entity rather than an attribute because exercises can use more than one muscle when performing the exercise (or in other words when training). Even though the datasets we found only have a single muscle or body part listed for each exercise, we didn't want users to be limited to assigning a single muscle to exercises they create in the future so we decided to plan ahead and model this relationship as a many to many relationship so we are not limited to supporting multi-muscle exercises (such as full body exercises) in the future. If we merged the Muscles entity into the Exercises entity an exercise would be limited to the number of muscles it can move by the number of columns we set aside to represent the Muscles entity attributes in the Exercises entity which would limit the number of exercises our application could correctly store.